APPLIED MACHINE LEARNING LAB

Experiment 1

(CSEG 1021)

SUBMITTED BY:

ALKA BISHT

SAP ID:590031136

SUBMITTED TO:

Dr. Pavinder Yadav

AssistantProfessor

House Price Prediction — Simple Linear Regression

Dataset: `Housing.csv` (columns: `area`, `price`)

This notebook walks through a complete, minimal pipeline:

1. Import data
2. Basic preprocessing
3. Visualize the data (scatter plot)
4. Train/test split
5. Train a Simple Linear Regression model
6. Report slope & intercept
7. Report accuracy metrics
8. Visualize the regression line
9. Predict price for a new area value

> **Note:** Since we're skipping feature management, this notebook uses a single feature (`area`) to predict `price`. If your CSV has more numeric columns, this same pipeline still works — just make sure `area` and `price` are present, or change the column names in Step 2.

Step 0: Install & Import Libraries

In [2]:
```
!pip install category_encoders

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.style.use('ggplot')

from scipy import stats

from sklearn.feature_selection import mutual_info_regression
from sklearn.model_selection import train_test_split
from category_encoders import MEstimateEncoder

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score

# Ignore Warnings
import warnings
warnings.filterwarnings("ignore")
```

```
Collecting category_encoders
  Downloading category_encoders-2.10.0-py3-none-any.whl.metadata (8.0 kB)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.13/dist-pac
kages (from category_encoders) (2.1.3)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.13/dist-pac
kages (from category_encoders) (2.2.3)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.13/dist-pack
ages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.13/di
st-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.13/dist-pack
ages (from category_encoders) (1.16.3)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.13/dis
t-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.1
3/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-pack
ages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-pa
ckages (from pandas>=1.0.5->category_encoders) (2026.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-pac
kages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/d
ist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.13/dist-p
ackages (from statsmodels>=0.9.0->category_encoders) (26.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages
(from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Downloading category_encoders-2.10.0-py3-none-any.whl (87 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 87.4/87.4 kB 1.1 MB/s eta 0:00:00
Installing collected packages: category_encoders
Successfully installed category_encoders-2.10.0
```

Step 1: Import the Data

On Google Colab, upload `Housing.csv` first. Run the cell below — it will open a file picker. (If you're running locally / the file is already in your working directory, you can skip the upload cell and just load it directly.)

In [9]:
```
# --- Uncomment this block if running on Google Colab and need to upload the file
# from google.colab import files
# uploaded = files.upload()   # select Housing.csv when prompted
# ------------------------------------------------------------------------------

# Load the dataset
excel_file = "/content/Housing.xlsx"
df = pd.read_excel(excel_file)
df.to_csv("Housing.csv", index=False)
# Quick look at the data
print("Shape:", df.shape)
print(df.head())
print(df.describe())
print(df.info())
```

```
Shape: (545, 13)
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
4              no             yes        2       no        furnished
              price          area    bedrooms   bathrooms     stories  \
count  5.450000e+02    545.000000  545.000000  545.000000  545.000000
mean   4.766729e+06   5150.541284    2.965138    1.286239    1.805505
std    1.870440e+06   2170.141023    0.738064    0.502470    0.867492
min    1.750000e+06   1650.000000    1.000000    1.000000    1.000000
25%    3.430000e+06   3600.000000    2.000000    1.000000    1.000000
50%    4.340000e+06   4600.000000    3.000000    1.000000    2.000000
75%    5.740000e+06   6360.000000    3.000000    2.000000    2.000000
max    1.330000e+07  16200.000000    6.000000    4.000000    4.000000

          parking
count  545.000000
mean     0.693578
std      0.861586
min      0.000000
25%      0.000000
50%      0.000000
75%      1.000000
max      3.000000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   price             545 non-null    int64
 1   area              545 non-null    int64
 2   bedrooms          545 non-null    int64
 3   bathrooms         545 non-null    int64
 4   stories           545 non-null    int64
 5   mainroad          545 non-null    object
 6   guestroom         545 non-null    object
 7   basement          545 non-null    object
 8   hotwaterheating   545 non-null    object
 9   airconditioning   545 non-null    object
 10  parking           545 non-null    int64
 11  prefarea          545 non-null    object
 12  furnishingstatus  545 non-null    object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
None
```

Step 2: Basic Data Preprocessing

We'll do the essentials only:

- Check for missing values
- Check data types
- Basic descriptive statistics
- Select the columns we need (`area` as feature, `price` as target)
- Drop duplicates / missing rows if any

In [11…

```
# Check info and missing values
print("\nMissing values per column:\n", df.isnull().sum())
```

```
Missing values per column:
 price               0
area                0
bedrooms            0
bathrooms           0
stories             0
mainroad            0
guestroom           0
basement            0
hotwaterheating     0
airconditioning     0
parking             0
prefarea            0
furnishingstatus    0
dtype: int64
```

In [12…

```
# Keep only the columns needed for this simple regression
data = df[['area', 'price']].copy()

# Drop any missing or duplicate rows
data = data.dropna()
data = data.drop_duplicates()

print("Final shape after cleaning:", data.shape)
data.head()
```

```
Final shape after cleaning: (532, 2)
```

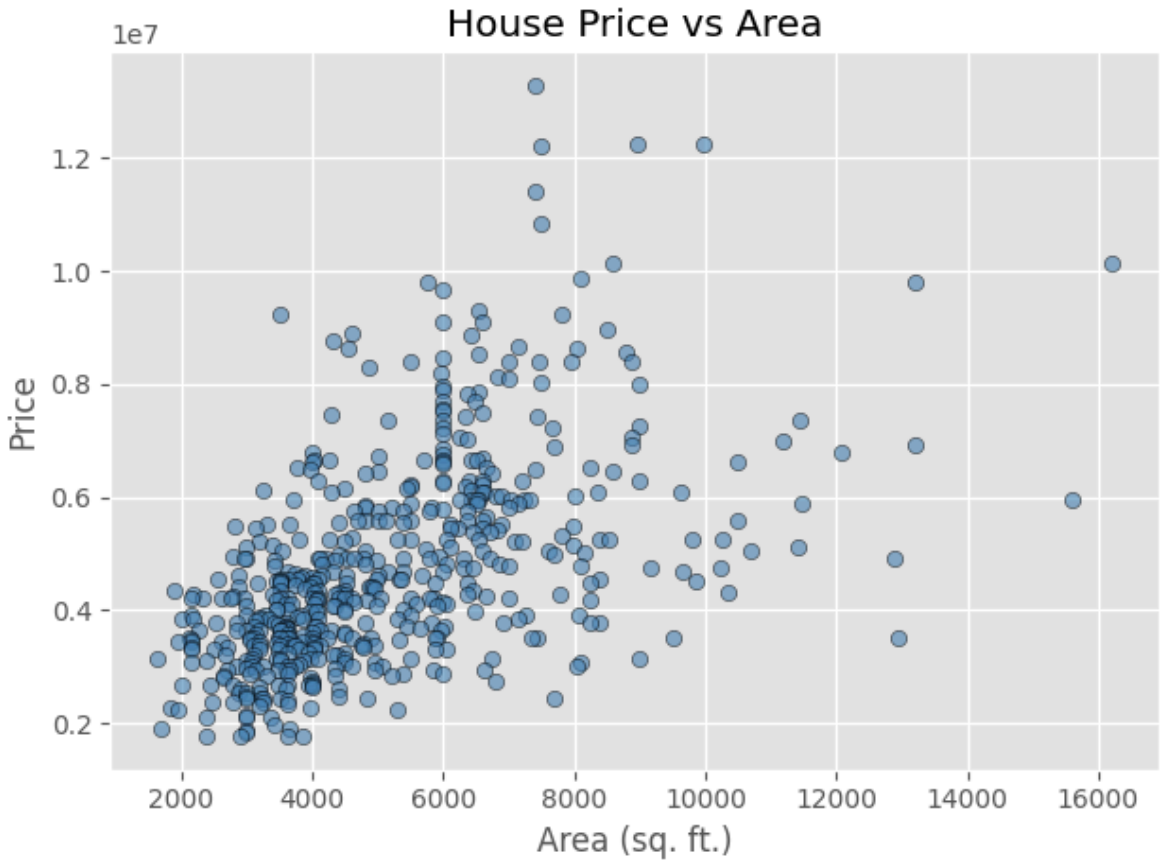
Out[12…

	area	price
0	7420	13300000
1	8960	12250000
2	9960	12250000
3	7500	12215000
4	7420	11410000

Step 3: Visualize the Data (Scatter Plot)

In [13…

```
import matplotlib.pyplot as plt

plt.scatter(data['area'], data['price'], color='steelblue', alpha=0.6, edgecolor='
plt.title('House Price vs Area')
plt.xlabel('Area (sq. ft.)')
plt.ylabel('Price')
plt.tight_layout()
plt.show()
```



Step 4: Train-Test Split

We split the data into training (80%) and testing (20%) sets.

In [14…

```
from sklearn.model_selection import train_test_split

X = data[['area']]   # Feature (kept as 2D DataFrame for sklearn)
y = data['price']     # Target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print("Training samples:", X_train.shape[0])
print("Testing samples :", X_test.shape[0])
print(X_train.head())
print(y_train.head())
```

```
Training samples: 425
Testing samples : 107
     area
306  4840
413  1950
458  3850
319  3000
145  5000
306    4165000
413    3430000
458    3115000
319    4060000
145    5600000
Name: price, dtype: int64
```

Step 5: Train the Linear Regression Model

In [15…

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print("Model trained successfully!")
```

```
Model trained successfully!
```

Step 6: Slope and Intercept

For simple linear regression: `price = slope * area + intercept`

In [16…

```
slope = model.coef_[0]
intercept = model.intercept_

print(f"Slope (coefficient)  : {slope:.4f}")
print(f"Intercept             : {intercept:.4f}")
print(f"\nEquation: price = {slope:.4f} * area + ({intercept:.4f})")
```

```
Slope (coefficient)  : 441.5876
Intercept             : 2450352.7349

Equation: price = 441.5876 * area + (2450352.7349)
```

Step 7: Accuracy Metrics

We evaluate the model on the **test set** using:

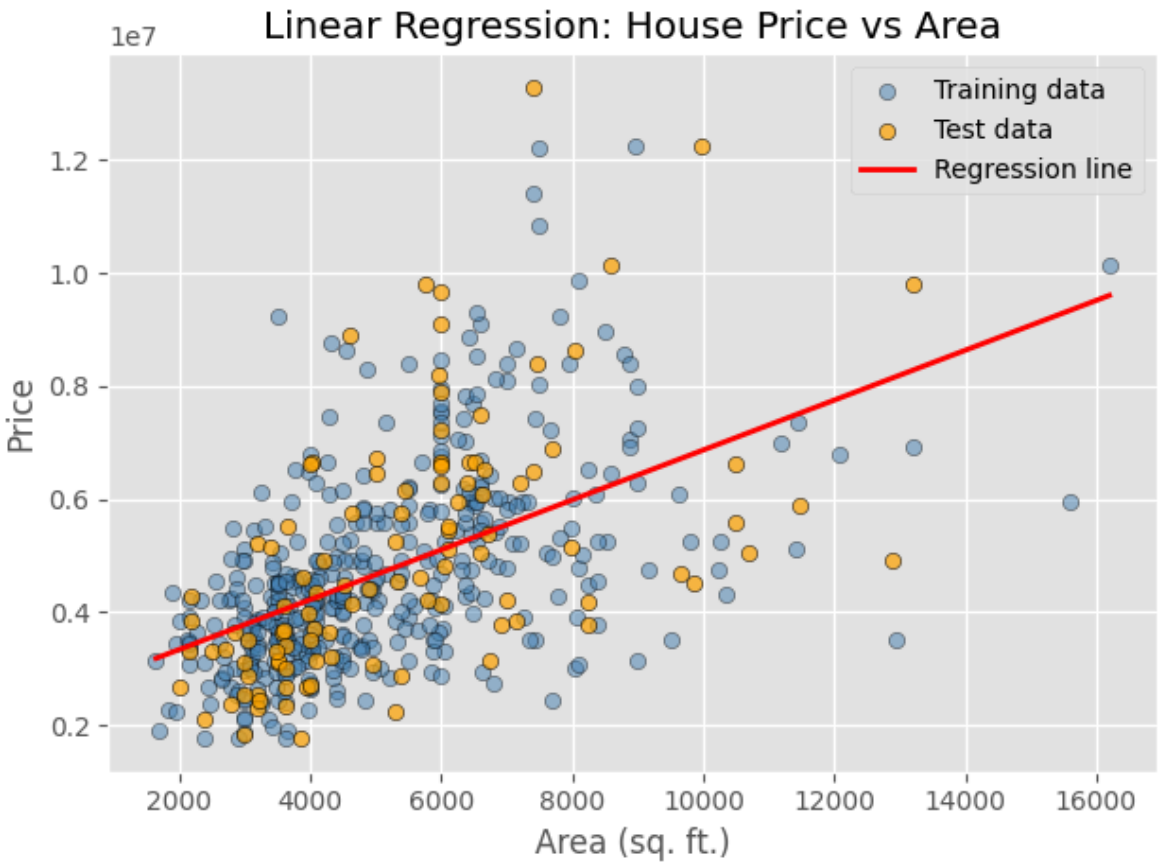
- **MAE** (Mean Absolute Error)
- **MSE** (Mean Squared Error)
- **RMSE** (Root Mean Squared Error)
- **R^2 Score** (coefficient of determination — how well the model explains variance in price)

In [17…
```
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error (MAE)  : {mae:,.2f}")
print(f"Mean Squared Error (MSE)   : {mse:,.2f}")
print(f"Root Mean Squared Error    : {rmse:,.2f}")
print(f"R² Score                   : {r2:.4f}")
```

```
Mean Absolute Error (MAE)  : 1,424,905.44
Mean Squared Error (MSE)   : 3,547,685,936,845.33
Root Mean Squared Error    : 1,883,530.18
R² Score                   : 0.2770
```

Step 8: Visualize the Regression Line

In [18…
```
plt.scatter(X_train, y_train, color='steelblue', alpha=0.5, edgecolor='k', label='
plt.scatter(X_test, y_test, color='orange', alpha=0.7, edgecolor='k', label='Test

# Regression line across the full range of area values
x_range = np.linspace(data['area'].min(), data['area'].max(), 100).reshape(-1, 1)
y_range_pred = model.predict(pd.DataFrame(x_range, columns=['area']))

plt.plot(x_range, y_range_pred, color='red', linewidth=2, label='Regression line')

plt.title('Linear Regression: House Price vs Area')
plt.xlabel('Area (sq. ft.)')
plt.ylabel('Price')
plt.legend()
plt.tight_layout()
plt.show()
```



Step 9: Predict Price for a New House

Use the trained model to predict the price of a house given its area.

In [19…

```
def predict_price(area_value):
    """Predict house price given an area (in sq. ft.) using the trained model."""
    input_df = pd.DataFrame([[area_value]], columns=['area'])
    predicted_price = model.predict(input_df)[0]
    return predicted_price

# Example usage
new_area = 3000   # change this value to any area you want to test
predicted = predict_price(new_area)
print(f"Predicted price for area = {new_area} sq. ft. : {predicted:,.2f}")
```

```
Predicted price for area = 3000 sq. ft. : 3,775,115.53
```

Summary

- Trained a simple linear regression model on `area → price`.
- Reported slope, intercept, and evaluation metrics (MAE, MSE, RMSE, R^2).
- Visualized both the raw data and the fitted regression line.
- Provided a reusable `predict_price()` function for new predictions.

In [ ]:

Open in Colab

House Price Prediction — Multiple Linear Regression

Dataset: `Housing.csv` (columns: `price`, `area`, `bedrooms`, `bathrooms`, `stories`, `mainroad`, `guestroom`, `basement`, `hotwaterheating`, `airconditioning`, `parking`, `prefarea`, `furnishingstatus`)

This is the multiple-feature version of the simple linear regression notebook: instead of predicting `price` from `area` alone, we use **all available columns** as features.

1. Import data
2. Basic preprocessing (encode categorical columns)
3. Visualize the data (correlation heatmap)
4. Train/test split
5. Train a Multiple Linear Regression model
6. Report coefficients & intercept
7. Report accuracy metrics
8. Visualize actual vs. predicted price
9. Predict price for a new house

> Note: `mainroad`, `guestroom`, `basement`, `hotwaterheating`, `airconditioning`, `prefarea` are yes/no columns, and `furnishingstatus` is a 3-level category — both need encoding before they can go into a linear model.

Step 0: Import Libraries

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('ggplot')

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Ignore Warnings
import warnings
warnings.filterwarnings("ignore")
```

Step 1: Import the Data

In [3]:
```
# --- Uncomment this block if running on Google Colab and need to upload the file
```

```
# from google.colab import files
# uploaded = files.upload()  # select Housing.csv when prompted
# ------------------------------------------------------------------------------

excel_file ="/content/Housing.xlsx"
df = pd.read_excel(excel_file)
df.to_csv("Housing.csv",index=False)
# Quick look at the data
print("Shape:", df.shape)
print(df.head())
print(df.info())
```

```
Shape: (545, 13)
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
4              no             yes        2       no        furnished
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   price             545 non-null    int64
 1   area              545 non-null    int64
 2   bedrooms          545 non-null    int64
 3   bathrooms         545 non-null    int64
 4   stories           545 non-null    int64
 5   mainroad          545 non-null    object
 6   guestroom         545 non-null    object
 7   basement          545 non-null    object
 8   hotwaterheating   545 non-null    object
 9   airconditioning   545 non-null    object
 10  parking           545 non-null    int64
 11  prefarea          545 non-null    object
 12  furnishingstatus  545 non-null    object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
None
```

Step 2: Basic Data Preprocessing

Since we're using every column this time, preprocessing needs to do a bit more than in the simple-regression notebook:

- Check for missing values
- Map the yes/no columns to 1/0
- One-hot encode `furnishingstatus` (3 categories → 2 dummy columns, dropping one level to avoid the dummy-variable trap)
- Drop duplicates / missing rows if any

In [4]:
```
# Check for missing values
print("Missing values per column:\n", df.isnull().sum())

data = df.copy()

# Map binary yes/no columns to 1/0
binary_cols = ['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'aircondit
for col in binary_cols:
    data[col] = data[col].map({'yes': 1, 'no': 0})

# One-hot encode furnishingstatus (drop_first avoids the dummy variable trap)
data = pd.get_dummies(data, columns=['furnishingstatus'], drop_first=True)

# Drop any missing or duplicate rows
data = data.dropna()
data = data.drop_duplicates()

print("Final shape after cleaning:", data.shape)
data.head()
```

```
Missing values per column:
 price               0
area                0
bedrooms            0
bathrooms           0
stories             0
mainroad            0
guestroom           0
basement            0
hotwaterheating     0
airconditioning     0
parking             0
prefarea            0
furnishingstatus    0
dtype: int64
Final shape after cleaning: (545, 14)
```

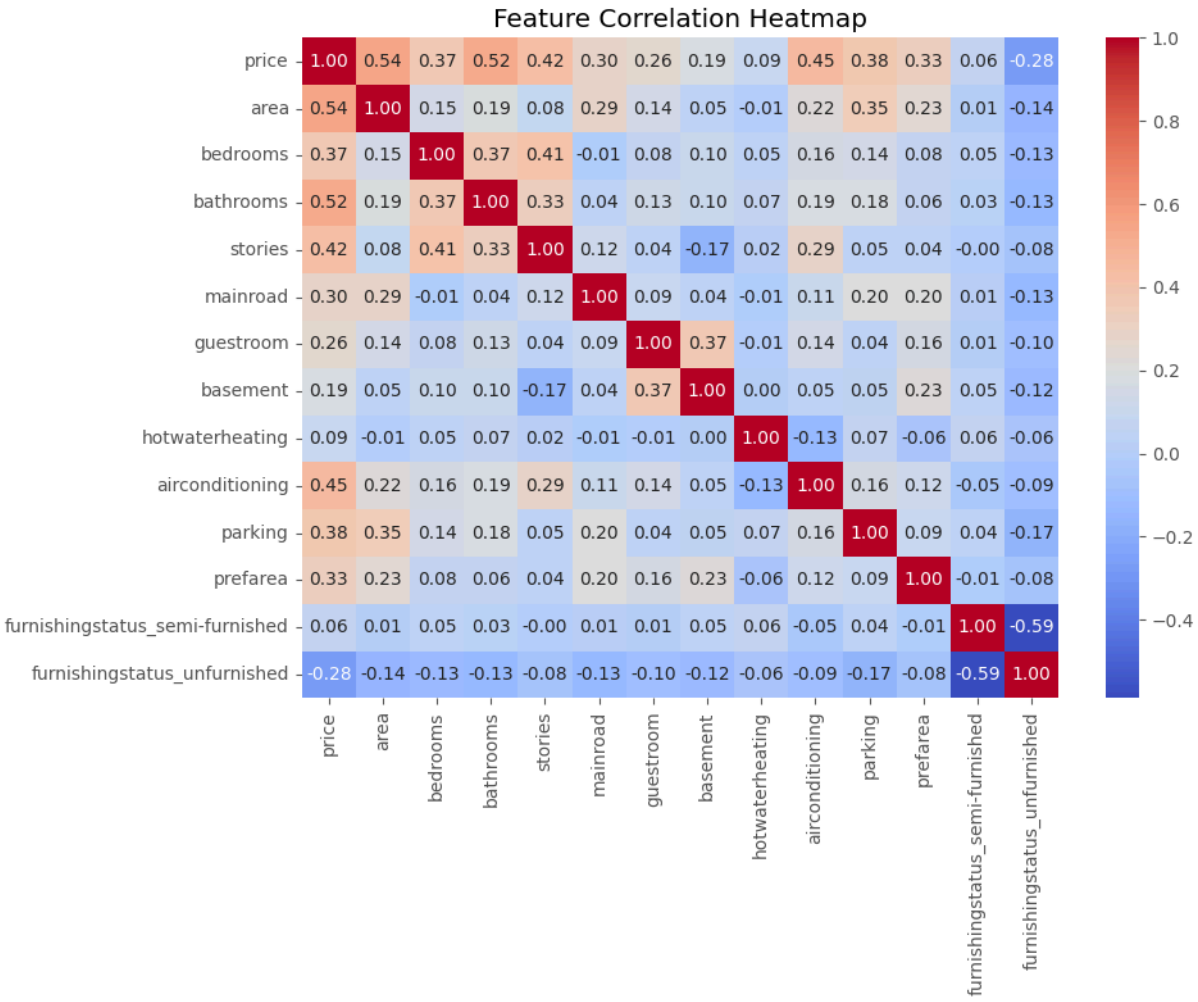
Out[4]:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotw
0	13300000	7420	4	2	3	1	0	0	
1	12250000	8960	4	4	4	1	0	0	
2	12250000	9960	3	2	2	1	0	1	
3	12215000	7500	4	2	2	1	0	1	
4	11410000	7420	4	1	2	1	1	1	

Step 3: Visualize the Data (Correlation Heatmap)

With a single feature a scatter plot was enough. With multiple features, a correlation heatmap is more useful for spotting which variables move with `price` — and which features are correlated with *each other* (a heads-up for multicollinearity).

In [5]:
```
plt.figure(figsize=(10, 8))
sns.heatmap(data.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.show()
```



Step 4: Train/Test Split

We split the data into training (80%) and testing (20%) sets. `X` now holds every column except `price`.

In [6]:
```
X = data.drop(columns=['price'])   # All features
y = data['price']                  # Target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print("Training samples:", X_train.shape[0])
print("Testing samples :", X_test.shape[0])
print("Features used   :", list(X.columns))
```

```
Training samples: 436
Testing samples : 109
Features used   : ['area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestro
om', 'basement', 'hotwaterheating', 'airconditioning', 'parking', 'prefarea', 'furn
ishingstatus_semi-furnished', 'furnishingstatus_unfurnished']
```

Step 5: Train the Multiple Linear Regression Model

In [7]:
```
model = LinearRegression()
model.fit(X_train, y_train)
print("Model trained successfully!")
```

```
Model trained successfully!
```

Step 6: Coefficients and Intercept

For multiple linear regression: `price = b1*x1 + b2*x2 + ... + bn*xn + intercept`

Each coefficient is the change in price for a one-unit increase in that feature, holding all other features constant.

In [8]:
```
coeffs = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
}).sort_values(by='Coefficient', key=abs, ascending=False)

print(coeffs.to_string(index=False))
print(f"\nIntercept: {model.intercept_:.4f}")
```

```
                      Feature   Coefficient
                    bathrooms  1.094445e+06
               airconditioning  7.914267e+05
               hotwaterheating  6.846499e+05
                     prefarea  6.298906e+05
  furnishingstatus_unfurnished -4.136451e+05
                      stories  4.074766e+05
                     basement  3.902512e+05
                     mainroad  3.679199e+05
                    guestroom  2.316100e+05
                      parking  2.248419e+05
furnishingstatus_semi-furnished -1.268818e+05
                     bedrooms  7.677870e+04
                         area  2.359688e+02

Intercept: 260032.3576
```

Step 7: Accuracy Metrics

Same metrics as before, plus **Adjusted R^2**, which corrects R^2 for the number of features used (useful once you're no longer down to a single predictor).

In [9]:
```
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

n = X_test.shape[0]
p = X_test.shape[1]
```

```
p = X_test.shape[1]
adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

print(f"Mean Absolute Error (MAE) : {mae:,.2f}")
print(f"Mean Squared Error (MSE)  : {mse:,.2f}")
print(f"Root Mean Squared Error   : {rmse:,.2f}")
print(f"R² Score                  : {r2:.4f}")
print(f"Adjusted R² Score         : {adj_r2:.4f}")
```

```
Mean Absolute Error (MAE) : 970,043.40
Mean Squared Error (MSE)  : 1,754,318,687,330.66
Root Mean Squared Error   : 1,324,506.96
R² Score                  : 0.6529
Adjusted R² Score         : 0.6054
```

Step 8: Visualize Actual vs. Predicted Price

With more than one feature we can't draw a single regression line against price. Instead: an **actual vs. predicted** scatter (points on the red line = perfect predictions) and a **residual plot** (looking for any leftover pattern, which would hint the linear model is missing something).

In [10]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Actual vs Predicted
axes[0].scatter(y_test, y_pred, color='steelblue', alpha=0.6, edgecolor='k')
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
axes[0].plot(lims, lims, color='red', linewidth=2, label='Perfect prediction')
axes[0].set_xlabel('Actual Price')
axes[0].set_ylabel('Predicted Price')
axes[0].set_title('Actual vs Predicted Price')
axes[0].legend()

# Residuals
residuals = y_test - y_pred
axes[1].scatter(y_pred, residuals, color='orange', alpha=0.6, edgecolor='k')
axes[1].axhline(0, color='red', linewidth=2)
axes[1].set_xlabel('Predicted Price')
axes[1].set_ylabel('Residual')
axes[1].set_title('Residual Plot')

plt.tight_layout()
plt.show()
```

Step 9: Predict Price for a New House

Use the trained model to predict the price of a house given its full feature set. The helper function applies the same encoding used during training and lines the columns up with what the model expects.

In [11]:
```
def predict_price(new_data: dict):
    """Predict house price given a dict of raw feature values, e.g.:
    {
        'area': 6000, 'bedrooms': 3, 'bathrooms': 2, 'stories': 2,
        'mainroad': 'yes', 'guestroom': 'no', 'basement': 'no',
        'hotwaterheating': 'no', 'airconditioning': 'yes',
        'parking': 1, 'prefarea': 'yes', 'furnishingstatus': 'semi-furnished'
    }
    """
    input_df = pd.DataFrame([new_data])

    for col in binary_cols:
        input_df[col] = input_df[col].map({'yes': 1, 'no': 0})

    input_df = pd.get_dummies(input_df, columns=['furnishingstatus'])

    # Align columns with training data (adds any missing dummy columns as 0)
    input_df = input_df.reindex(columns=X.columns, fill_value=0)

    predicted_price = model.predict(input_df)[0]
    return predicted_price

# Example usage
new_house = {
    'area': 6000, 'bedrooms': 3, 'bathrooms': 2, 'stories': 2,
    'mainroad': 'yes', 'guestroom': 'no', 'basement': 'no',
    'hotwaterheating': 'no', 'airconditioning': 'yes',
    'parking': 1, 'prefarea': 'yes', 'furnishingstatus': 'semi-furnished'
}
predicted = predict_price(new_house)
print(f"Predicted price for new house: {predicted:,.2f}")
```

```
Predicted price for new house: 6,797,221.40
```

Summary

- Trained a multiple linear regression model on all 12 features → `price`.
- Encoded yes/no columns as 1/0 and one-hot encoded `furnishingstatus`.
- Reported per-feature coefficients, the intercept, and evaluation metrics (MAE, MSE, RMSE, R^2, Adjusted R^2).
- Visualized actual-vs-predicted price and residuals (no single 2D regression line, since there are multiple predictors).
- Provided a reusable `predict_price()` function that takes a dict of raw feature values and handles the encoding for you.

In [ ]:
```
```

House Price Prediction — Polynomial Regression (Degrees 1–15)

Dataset: `Housing.csv` (columns: `area`, `price`)

This extends the simple linear regression notebook by fitting **polynomial** models of increasing degree to the same `area → price` relationship, and comparing how they generalize.

1. Import data
2. Basic preprocessing
3. Visualize the data (scatter plot)
4. Train/test split
5. Train polynomial regression models for degrees 1 through 15
6. Report metrics for every degree
7. Visualize the bias–variance tradeoff (train vs. test error by degree)
8. Visualize the fitted curves for a few representative degrees
9. Pick the best degree and predict price for a new area value

> A degree-1 polynomial *is* simple linear regression. As the degree increases, the model gets more flexible — up to a point where it starts fitting noise instead of signal (overfitting). This notebook makes that tradeoff visible.

Step 0: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.style.use('ggplot')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline, Pipeline # Explicitly import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.compose import ColumnTransformer # Import ColumnTransformer

# Ignore Warnings
import warnings
warnings.filterwarnings("ignore")
```

Step 1: Import the Data

```
# --- Uncomment this block if running on Google Colab and need to upload the file ---
# from google.colab import files
# uploaded = files.upload()  # select Housing.csv when prompted
# ------------------------------------------------------------------------------------
# Google Colab upload
from google.colab import files

uploaded = files.upload()

# Read CSV
excel_file = "/content/Housing.xlsx"
df = pd.read_excel(excel_file)
df.to_csv("Housing.csv", index=False)


df = pd.read_csv("Housing.csv")
print("Dataset loaded successfully.")
print("Shape:", df.shape)
print(df.head())
```

```
Choose files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Housing.xlsx to Housing (1).xlsx
Dataset loaded successfully.
Shape: (545, 13)
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
```

Step 2: Basic Data Preprocessing

Same essentials as the simple linear regression notebook: keep `area` and `price`, drop missing/duplicate rows.

```
print("Column names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())
```

```
Column names:
['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioni

Data types:
price              int64
area               int64
bedrooms           int64
bathrooms          int64
stories            int64
mainroad          object
guestroom         object
basement          object
hotwaterheating   object
airconditioning   object
parking            int64
prefarea          object
furnishingstatus  object
dtype: object

Missing values:
price             0
area              0
bedrooms          0
bathrooms         0
stories           0
mainroad          0
guestroom         0
basement          0
hotwaterheating   0
airconditioning   0
parking           0
prefarea          0
furnishingstatus  0
dtype: int64

Duplicate rows: 0
```

```
# Keep only the columns required for this model
data = df[["area","bedrooms", "bathrooms", "stories", "parking", "price"]].copy()

# Convert to numeric in case the CSV contains text-formatted numbers
data["area"] = pd.to_numeric(data["area"], errors="coerce")
data["price"] = pd.to_numeric(data["price"], errors="coerce")

# Remove missing and duplicate records
data = data.dropna()
data = data.drop_duplicates()

print("Shape after preprocessing:", data.shape)
```

```
print("\nMissing values after preprocessing:")
print(data.isnull().sum())

data.head()

# Define X_PLR and y_PLR from the cleaned 'data' DataFrame
X_PLR = data[["area", "bedrooms", "bathrooms", "stories", "parking"]]
y_PLR = data["price"]
```

```
Shape after preprocessing: (544, 6)

Missing values after preprocessing:
area         0
bedrooms     0
bathrooms    0
stories      0
parking      0
price        0
dtype: int64
```

Step 3: Select features and target

```
X_PLR = df[["area", "bedrooms", "bathrooms", "stories", "parking"]]
y_PLR = df["price"]
```

Step 4: Train/Test Split

We split once, up front, and reuse the same split for every degree so the comparison across degrees is fair.

```
X_PLR_train, X_PLR_test, y_PLR_train, y_PLR_test = train_test_split(
    X_PLR,
    y_PLR,
    test_size=0.20,
    random_state=42
)
```

```
print("X_PLR_train:", X_PLR_train.shape)
print("X_PLR_test:", X_PLR_test.shape)
print("y_PLR_train:", y_PLR_train.shape)
print("y_PLR_test:", y_PLR_test.shape)
```

```
X_PLR_train: (436, 5)
X_PLR_test: (109, 5)
y_PLR_train: (436,)
y_PLR_test: (109,)
```

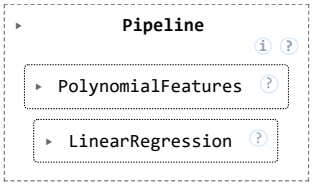
Step 5: Build the Polynomial pipeline

For each degree we build a small pipeline:

1. `PolynomialFeatures(degree)` — expands `area` into `area, area², area³, ..., area^degree`
2. `StandardScaler()` — rescales those terms to comparable ranges. Without this, `area^15` for a 16,000 sq. ft. house is an astronomically large number, and `LinearRegression` becomes numerically unstable trying to fit it.
3. `LinearRegression()` — fits ordinary least squares on the (scaled) polynomial terms

We fit one pipeline per degree and keep every model, so we can compare and re-visualize them afterward.

```
degree = 3  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

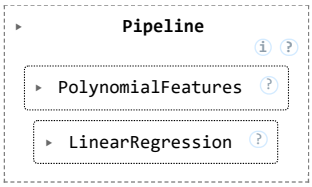



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 3
MAE:  1189493.38
MSE:  2959403787091.53
RMSE: 1720291.77
R²:   0.415
```

```
degree = 4  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

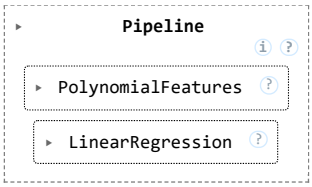



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 4
MAE:  1117755.85
MSE:  2391814713435.19
RMSE: 1546549.29
R²:   0.527
```

```
degree = 5  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

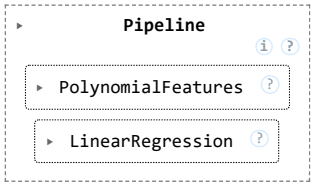



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 5
MAE:  1207191.72
MSE:  2709318149201.01
RMSE: 1646000.65
R²:   0.464
```

```
degree = 6  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

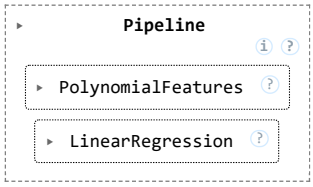



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 6
MAE:  1250163.36
MSE:  3480083921381.68
RMSE: 1865498.30
R²:   0.311
```

```
degree = 7  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```



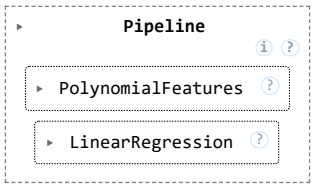

```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 7
MAE:  1316517.88
MSE:  3605288249542.39
RMSE: 1898759.66
R²:   0.287
```

```
degree = 8  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
```

```
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

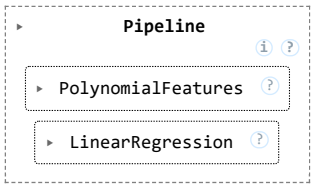



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 8
MAE:  1830119.59
MSE:  15095064606515.77
RMSE: 3885236.75
R²:   -1.986
```

```
degree = 9  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

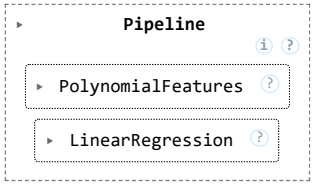



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 9
MAE:  2123146.41
MSE:  32233818031832.38
RMSE: 5677483.42
R²:   -5.377
```

```
degree = 10  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

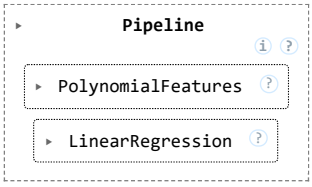


```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 10
MAE:  2514279.03
MSE:  69693771182546.79
RMSE: 8348279.53
R²:   -12.788
```

```
degree = 11  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

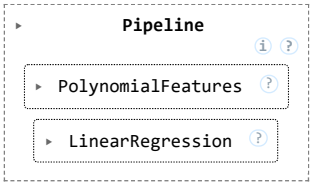



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 11
MAE:  3012598.36
MSE:  149275317415573.97
RMSE: 12217827.85
R²:   -28.533
```

```
degree = 12  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```



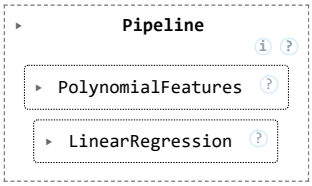

```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
```

```
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 12
MAE:  3668921.01
MSE:  314856065807181.06
RMSE: 17744184.00
R²:   -61.291
```

```
degree = 13  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```



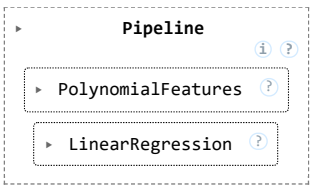

```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

Double-click (or enter) to edit

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 13
MAE:  3988794.99
MSE:  477023702205048.38
RMSE: 21840872.29
R²:   -93.375
```

```
degree = 14  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```

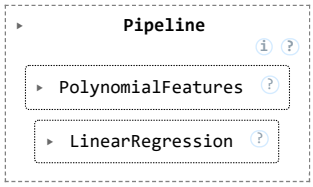



```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 14
MAE:  3568721.74
MSE:  367514272938772.75
RMSE: 19170661.78
R²:   -71.709
```

```
degree = 15  # try 2, 3, 4... and compare
model_PLR = make_pipeline(
    PolynomialFeatures(degree=degree, include_bias=False),
    LinearRegression()
)
model_PLR.fit(X_train_PLR, y_train_PLR)
```




```
y_pred_PLR = model_PLR.predict(X_test_PLR)
```

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 15
MAE:  2889812.31
MSE:  179799735579870.59
RMSE: 13408942.37
R²:   -34.572
```

Double-click (or enter) to edit

Step 6: Report Metrics for Every Degree

Watch **Train RMSE** keep dropping as degree increases (the model fits the training data ever more closely) while **Test RMSE** stops improving — and eventually gets worse — once the model starts overfitting.

```
results = []
models = {} # Store models for later use

poly_features_col = ['area']
passthrough_features_cols = ['bedrooms', 'bathrooms', 'stories', 'parking']

for degree in range(1, 16):

    # Define the preprocessor using ColumnTransformer
    # It applies PolynomialFeatures and then scales the output of poly_features_col
    # And applies StandardScaler to the passthrough_features_cols
    preprocessor = ColumnTransformer(
        transformers=[
            ('poly_and_scale_area', Pipeline([
                ('poly_feat', PolynomialFeatures(degree=degree, include_bias=False)),
                ('scaler', StandardScaler())
            ]), poly_features_col), # Apply to 'area'
            ('scale_others', StandardScaler(), passthrough_features_cols) # Scale other features
        ],
        remainder='passthrough', # Handle any other columns not explicitly listed
        verbose_feature_names_out=False # Critical for keeping original feature names for downstream estimator
    )

    # Create the full pipeline
    model = Pipeline([
        ('preprocessor', preprocessor),
        ('regressor', LinearRegression())
    ])

    model.fit(X_PLR_train, y_PLR_train)

    y_train_pred = model.predict(X_PLR_train)
    y_test_pred = model.predict(X_PLR_test)

    train_rmse = np.sqrt(mean_squared_error(y_PLR_train, y_train_pred))
    test_rmse = np.sqrt(mean_squared_error(y_PLR_test, y_test_pred))
    train_r2 = r2_score(y_PLR_train, y_train_pred)
```

```
    test_r2 = r2_score(y_PLR_test, y_test_pred)
    test_mae = mean_absolute_error(y_PLR_test, y_test_pred)

    results.append({
        'Degree': degree,
        'Train RMSE': train_rmse,
        'Test RMSE': test_rmse,
        'Train R2': train_r2,
        'Test R2': test_r2,
        'Test MAE': test_mae
    })
    models[degree] = model

results_df = pd.DataFrame(results)

# This cell now produces the comprehensive results_df for plotting and analysis
print(results_df)
```

```
    Degree    Train RMSE     Test RMSE  Train R2   Test R2      Test MAE
0        1  1.161899e+06  1.514174e+06  0.562168  0.546406  1.127483e+06
1        2  1.136958e+06  1.511519e+06  0.580763  0.547995  1.116812e+06
2        3  1.134882e+06  1.508498e+06  0.582292  0.549800  1.108611e+06
3        4  1.118835e+06  1.545588e+06  0.594021  0.527389  1.142476e+06
4        5  1.117733e+06  1.538673e+06  0.594820  0.531609  1.140305e+06
5        6  1.116636e+06  1.539074e+06  0.595615  0.531365  1.143300e+06
6        7  1.113048e+06  1.540676e+06  0.598210  0.530389  1.144299e+06
7        8  1.111674e+06  1.539266e+06  0.599201  0.531248  1.145524e+06
8        9  1.110125e+06  1.548630e+06  0.600318  0.525527  1.149521e+06
9       10  1.108728e+06  1.560232e+06  0.601323  0.518392  1.152775e+06
10      11  1.106025e+06  1.536762e+06  0.603265  0.532772  1.136662e+06
11      12  1.103254e+06  1.528273e+06  0.605250  0.537919  1.134790e+06
12      13  1.101269e+06  1.526190e+06  0.606669  0.539178  1.132504e+06
13      14  1.096147e+06  1.531574e+06  0.610319  0.535921  1.134211e+06
14      15  1.096064e+06  1.533879e+06  0.610378  0.534523  1.134720e+06
```

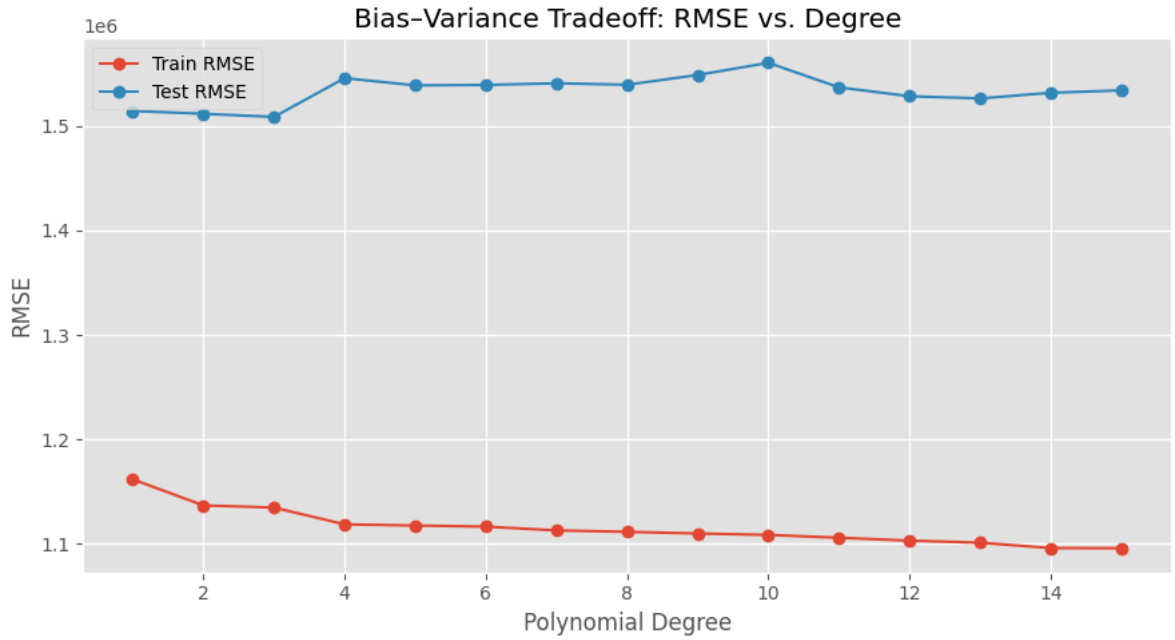
```
results_PLR = pd.DataFrame(
    results_PLR,
    columns=[
        "Degree",
        "Test RMSE",
        "Test R2"
    ]
)

results_PLR
```

	Degree	Test RMSE	Test R2
0	2	1.525169e+06	0.539795
1	3	1.720292e+06	0.414509
2	4	1.546549e+06	0.526802
3	5	1.646001e+06	0.463986
4	6	1.865498e+06	0.311498
5	7	1.898760e+06	0.286727
6	8	3.885237e+06	-1.986419
7	9	5.677483e+06	-5.377163
8	10	8.348280e+06	-12.788268
9	11	1.221783e+07	-28.532742
10	12	1.774418e+07	-61.291362
11	13	2.184087e+07	-93.374730
12	14	1.917066e+07	-71.709302
13	15	1.340894e+07	-34.571716

Step 7: Visualize the Bias–Variance Tradeoff

Plotting train vs. test error against degree makes the tradeoff visible: error falls for both as the model gains useful flexibility, then train and test error diverge once the model starts memorizing noise.

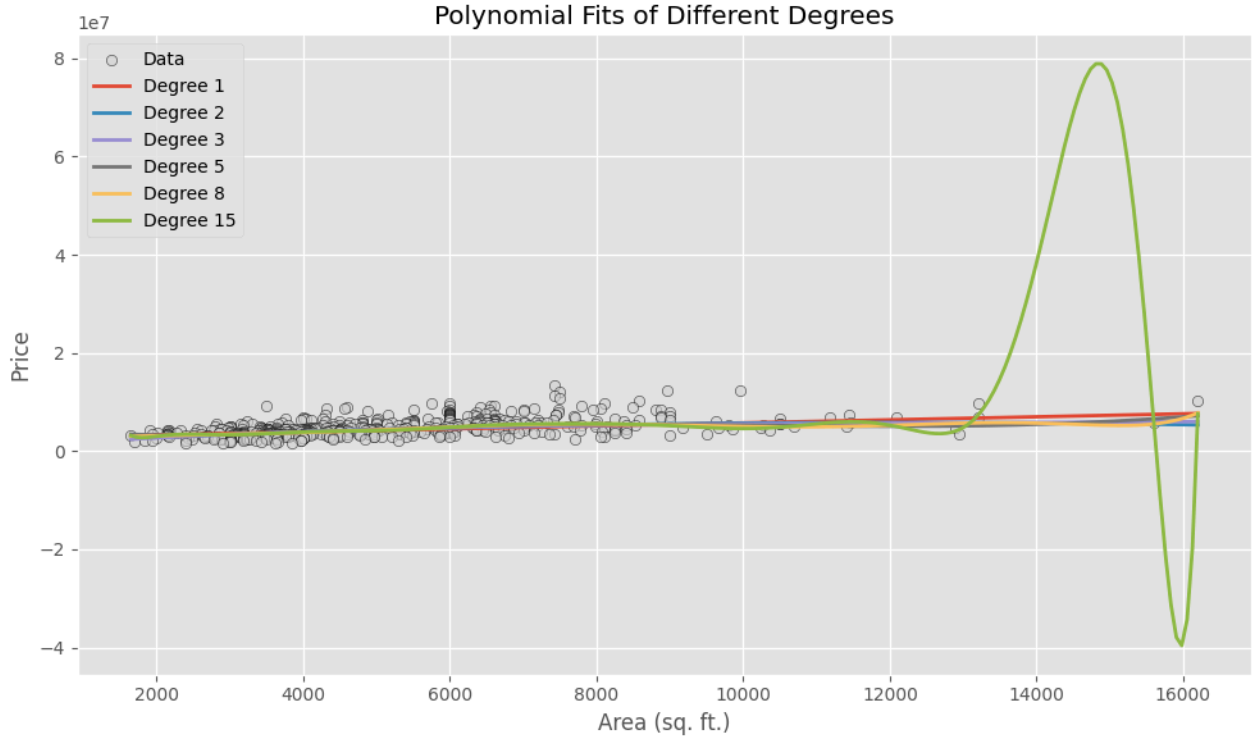
```
plt.figure(figsize=(9, 5))
plt.plot(results_df['Degree'], results_df['Train RMSE'], marker='o', label='Train RMSE')
plt.plot(results_df['Degree'], results_df['Test RMSE'], marker='o', label='Test RMSE')
plt.xlabel('Polynomial Degree')
plt.ylabel('RMSE')
plt.title('Bias-Variance Tradeoff: RMSE vs. Degree')
plt.legend()
plt.tight_layout()
plt.show()
```



Step 8: Visualize the Fitted Curves

Overlaying a few representative degrees on the scatter plot shows the shape each model actually learned — low degrees underfit with a straight (or gently curved) line, while very high degrees can start to wiggle, especially near the edges of the data.

```
degrees_to_plot = [1, 2, 3, 5, 8, 15]

x_range = np.linspace(data['area'].min(), data['area'].max(), 200)

# Create a DataFrame for prediction with all features
# Set other features to their median values from the training data for a representative plot
x_plot_df = pd.DataFrame(columns=X_PLR.columns) # Start with all column names
x_plot_df['area'] = x_range

# Fill other columns with median values from the training set
for col in ["bedrooms", "bathrooms", "stories", "parking"]:
    x_plot_df[col] = X_PLR_train[col].median()

plt.figure(figsize=(10, 6))
plt.scatter(data['area'], data['price'], color='lightgray', alpha=0.6, edgecolor='k', label='Data')

for d in degrees_to_plot:
    y_curve = models[d].predict(x_plot_df)
    plt.plot(x_range, y_curve, linewidth=2, label=f'Degree {d}')

plt.xlabel('Area (sq. ft.)')
plt.ylabel('Price')
plt.title('Polynomial Fits of Different Degrees')
plt.legend()
plt.tight_layout()
plt.show()
```



Step 9: Pick the Best Degree and Predict

We select the degree with the lowest **test** RMSE (not train RMSE — a low train error alone just means the model memorized the training data) and use that model to predict on a new area value.

A caution on high-degree extrapolation: predicting *inside* the range of areas the model was trained on is fine. Predicting *outside* that range with a high-degree polynomial can produce wildly unrealistic results, because the curve is free to swing sharply beyond the last data point. The check below demonstrates this directly.

```
mae = mean_absolute_error(y_test_PLR, y_pred_PLR)
mse = mean_squared_error(y_test_PLR, y_pred_PLR)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_PLR, y_pred_PLR)

print(f"Degree: {degree}")
print(f"MAE:  {mae:.2f}")
print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
```

```
Degree: 15
MAE:  2889812.31
MSE:  179799735579870.59
RMSE: 13408942.37
R²:   -34.572
```

```
# Find the degree with the lowest Test RMSE
best_degree = results_df.loc[results_df['Test RMSE'].idxmin()]['Degree']
best_model = models[best_degree]

print(f"The best degree based on lowest Test RMSE is: {int(best_degree)}")
```

```
The best degree based on lowest Test RMSE is: 3
```

Step 10: Let's try predicting for an area far outside the training range, e.g., 20250 sq. ft.

For demonstration, we'll use a high degree model (e.g., degree 15) and the best model for comparison.

```
# Let's try predicting for an area far outside the training range, e.g., 20250 sq. ft.
# For demonstration, we'll use a high degree model (e.g., degree 15) and the best model for comparison.

# Ensure the column names and order match the training data (X_PLR)
far_area_data = [[20250, 3, 2, 2, 1]]
far_area = pd.DataFrame(far_area_data, columns=X_PLR.columns)

# Using the highest degree model available (degree 15)
high_degree_model = models[15] # Assuming 15 is the highest degree trained
high_degree_pred = high_degree_model.predict(far_area)

# Using the best degree model
low_degree_pred = best_model.predict(far_area)

print(f"Predicted price for {far_area['area'].iloc[0]} sq. ft. using Degree 15 model: ${high_degree_pred[0]:,.2f}")
print(f"Predicted price for {far_area['area'].iloc[0]} sq. ft. using Best Degree ({int(best_degree)}) model: ${low_degree_p

# Note: The predictions will likely be wildly different, especially for the high-degree model,
# illustrating the danger of extrapolation with high-degree polynomials.
```

```
Predicted price for 20250 sq. ft. using Degree 15 model: $1,497,353,928,109.17
Predicted price for 20250 sq. ft. using Best Degree (3) model: $8,567,349.22
```

Predict for a new area

```
# Example: Predict price for a 3000 sq. ft. area
# Ensure the column names and order match the training data (X_PLR)
new_area_data = [[3000, 3, 2, 2, 1]]
new_area = pd.DataFrame(new_area_data, columns=X_PLR.columns)
predicted = best_model.predict(new_area)

print(f"Predicted price for a {new_area['area'].iloc[0]} sq. ft. house (using best model degree {int(best_degree)}): ${pred
```

```
Predicted price for a 3000 sq. ft. house (using best model degree 3): $4,867,136.84
```

Summary

- Trained polynomial regression models on `area → price` for every degree from 1 to 15, using the same train/test split throughout.
- Scaled polynomial terms with `StandardScaler` to keep high-degree fits numerically stable.
- Reported train/test RMSE, R^2, and test MAE for each degree.
- Visualized the bias–variance tradeoff: train error keeps falling while test error eventually flattens or worsens.
- Visualized the actual fitted curves for several degrees side by side.
- Picked the degree with the lowest test RMSE and used it for prediction, with a demonstration of why extrapolating with a high-degree model is risky.